Geometrically Accurate Gaussian Avatars

Sotirios Karapiperis* Emircan Gündoğdu* Brian Barth*
ETH Zürich
{skarapiperis, egundogdu, bbarth}@student.ethz.ch

Abstract

Reconstructing animatable human avatars from monocular video is a challenging task, particularly in achieving accurate surface geometry. Recent progress in Gaussian Splatting has enabled fast, differentiable rendering of 3D scenes and deformable models, making it a promising tool for neural avatar reconstruction. In this project, we build on a 3DGS-avatar baseline and investigate how more geometry-focused extensions, such as 2D Gaussian Splatting and Gaussian Opacity Fields, can improve mesh quality and surface reconstruction. Additionally, we experiment with different depth rendering strategies and mesh extraction techniques to assess their effect on reconstruction accuracy. We evaluate the performance of these methods using both image-based and geometry-based metrics on the ZJU-MoCap dataset. The source code can be found at: https://gitlab.ethz.ch/skarapiperis/dh25

1. Introduction

Reconstructing realistic and animatable digital humans from monocular video is a core challenge in computer vision and graphics. Accurate digital avatars have broad applications in virtual reality, telepresence, and content creation, where both visual realism and geometric fidelity are essential.

Recent methods based on Gaussian Splatting [5] have shown great promise by enabling real-time rendering of scenes using collections of anisotropic 3D Gaussians. These representations support differentiable rendering and efficient training, and have been extended to human avatars [4,6,8] via learned deformation models. Despite strong rendering performance, questions remain about how well these methods capture fine-scale geometry, and how reconstruction quality varies across different Gaussian formulations.

This project investigates how geometry-focused improvements to Gaussian-based rendering can enhance the quality of avatar reconstruction from monocular video. We take a modular approach: starting from a baseline Gaussian avatar framework [8], we integrate and compare recent methods that explicitly target geometric accuracy. These include surface-aligned Gaussians [3, 10] and regularization strategies aimed at improving mesh extraction.

Our objectives are twofold. First, we evaluate whether replacing the standard Gaussian representation in the baseline pipeline with more geometry-aware alternatives leads to improved reconstruction. Second, we qualitatively assess the effects of these changes on the reconstructed avatars, identifying success cases and remaining challenges in the pursuit of high-fidelity geometry. Our contributions are as follows:

- We explore and compare different depth estimation methods and their effects on mesh extraction.
- We integrate two recent alternatives to the standard 3DGS representation, 2D Gaussian Splatting and Gaussian Opacity Fields, with the goal of producing more geometrically accurate meshes.

2. Related Work

2.1. 3D Gaussian Splatting (3DGS)

Gaussian Splatting [5] is a 3D rendering technique that represents scenes using a collection of anisotropic 3D Gaussian kernels—also referred to as "splats"—instead of traditional polygonal meshes or point clouds. Each Gaussian is defined by a spatial position $\mathbf{p}_k \in \mathbb{R}^3$, a shape matrix $S_k \in \mathbb{R}^{3\times3}$, a rotation matrix $\mathbf{R}_k \in \mathbb{R}^{3\times3}$, as well as color and opacity parameters. This formulation enables a continuous volumetric representation of geometry and appearance.

A key advantage of Gaussian Splatting is its differentiable rasterization process. Unlike traditional mesh-based rasterization, which is typically non-differentiable, the rendering pipeline of Gaussian Splatting supports backpropagation, making it suitable for optimization-based reconstruction tasks. Moreover, it is computationally efficient and achieves real-time performance on modern GPUs.

In our work, 3D Gaussian Splatting serves as the baseline representation for reconstructing animatable human avatars

*All authors contributed equally to this work

from monocular video. Its differentiable nature allows us to optimize both geometry and appearance directly from image data while maintaining high-quality rendering performance.

2.2. 2D Gaussian Splatting (2DGS)

An alternative to 3DGS is 2D Gaussian Splatting (2DGS) [3], which replaces 3D ellipsoidal primitives with surface-aligned 2D Gaussian disks. Each 2D splat is parameterized by a center point $\mathbf{p}_k$ and two orthogonal tangent vectors $\mathbf{t}_u$ and $\mathbf{t}_v$, which define a local surface-aligned frame. The scaling parameters s_u and s_v control the Gaussian spread along each tangent direction. Together, they define the point in space for coordinates (u, v) as:

$$P(u, v) = \mathbf{p}_k + s_u \mathbf{t}_u u + s_v \mathbf{t}_v v$$

For compatibility with 3DGS infrastructure, we can reinterpret the local frame by constructing a rotation matrix R whose first two columns are $\mathbf{t}_u$ and $\mathbf{t}_v$, and whose third column is their cross product—effectively, the disk normal. The scaling parameters can similarly be embedded into a 3×3 diagonal matrix, with the third diagonal entry set to zero.

The value of the Gaussian at point (u, v) is computed as:

$$G(u, v) = \exp\left(-\frac{u^2 + v^2}{2}\right)$$

As in 3DGS, each 2D Gaussian has an associated opacity and texture feature. However, the rasterization process differs: rather than projecting 3D Gaussians into image space, the 2DGS method computes splat-ray intersections in the reverse direction. For each pixel, the method determines whether a ray intersects a 2D Gaussian disk, and then computes the corresponding (u, v) coordinates in the disk's local frame. The image is formed using alpha blending, where the alpha value is given by the product of the opacity and the Gaussian value $G(u, v)$.

This surface-aware formulation improves alignment with object geometry and has been shown to improve mesh extraction quality over standard 3DGS.

2.3. Gaussian Opacity Fields

Gaussian Opacity Fields (GOF) [10] aim to enhance 3D scene reconstruction by improving mesh and surface extraction directly from Gaussian-based volumetric representations. Traditional mesh extraction methods often produce artifacts and inconsistent geometry.

GOF introduces a novel way to extract surfaces by using opacity fields derived from a set of 3D Gaussians. Instead of projecting Gaussians to 2D image space as in the classical 3D Gaussian splatting, GOF explicitly takes rays through the 3D volume into consideration. This ray-based method allows the computation of opacity values for any 3D point, yielding a consistent field across views and avoiding reliance on external fusion techniques like Truncated Signed Distance Function (TSDF) [11]. The main idea here is the preserve 3D information by defining ray-Gaussian intersections and evaluating Gaussian contributions along those rays.

Specifically, a ray is defined by a camera origin $\mathbf{o} \in \mathbb{R}^3$ and a direction vector $\mathbf{r} \in \mathbb{R}^3$. Any point along this ray is parameterized as $x(t) = o + tr$, where t denotes the depth along the ray. To be able to evaluate the contribution of a Gaussian centered at $\mathbf{p}_k$ with rotation matrix R_k and scaling matrix S_k, the ray is transformed into the local coordinate system of the Gaussian as with new camera origin, $\mathbf{o}_g$, new ray direction, $\mathbf{r}_g$ as:

$$\mathbf{o}_g = S_k^{-1}\mathbf{R_k}(\mathbf{o} - \mathbf{p}_k), \qquad \mathbf{r}_g = S_k^{-1}\mathbf{R_k}\mathbf{r}.$$

Then any point along the ray becomes $\mathbf{x}_g(t) = \mathbf{o}_g + t\mathbf{r}_g$. The Gaussian value along the ray becomes a 1D Gaussian function as:

$$G_k^{1D}(t) = e^{-\frac{1}{2}\mathbf{x}_g(t)^\top \mathbf{x}_g(t)}.$$

The contribution of a Gaussian is then evaluated at the depth t along the ray where this function reaches its maximum value.

2.4. Gaussian Avatars

Gaussian avatars [6–8] leverage the Gaussian splatting by modeling a human as a collection of Gaussian primitives that encode geometry and appearance. Each primitive carries spatial, opacity, and color information that, when projected via Gaussian splatting, forms a smooth and continuous surface representation. Unlike traditional mesh models, the continuous nature of Gaussians allows for naturally smooth surfaces without explicit polygons, reducing artifacts such as faceting or sharp edges.

This smooth representation also facilitates flexible animation and deformation, as the Gaussian primitives can be transformed coherently with underlying skeletal or blendshape parameters to produce natural motion while maintaining geometric consistency [8].

Moreover, Gaussian avatars benefit from the differentiable rasterization enabled by Gaussian splatting. This differentiability is crucial for optimizing geometry and appearance in scenarios with limited or indirect supervision.

In addition, by embedding color and opacity within each Gaussian primitive, these avatars can effectively capture complex appearance effects such as translucency, soft shadows, and subtle surface details, which are often difficult to represent with traditional mesh texturing techniques.

By combining these strengths, Gaussian avatars provide a compact, flexible, and expressive model for animatable digital humans.

3. Method

Our method builds upon the popular Gaussian Avatar method 3DGS-avatar [8]. In the following section we briefly describe the method, highlighting parts that are of importance to our method.

3.1. 3DGS-Avatar

The method is illustrated in 1. The inputs to the pipeline are a monocular video of the human and the pre-estimated SMPL parameters(shape,pose) for each frame. Each gaussian is described by its position, scale, rotation, opacity and additionaly it carries a texture feature. The gaussians, that originaly lie in the canonical space are deformed in the observation space and then rendered to the image space in a diffenrentiable manner. The deformation happens in two phases. Firstly a non-rigid deformer f_{nr}, parameterized by a neural network, predicts for each gaussian an offset for its position, scale, rotation and a texture feature fector given as input the canonical gaussians and a latent pose feature that:

$$(\delta\mathbf{x}, \delta\mathbf{s}, \delta\mathbf{q}, \mathbf{z}) = f_{nr}(x_c; Z_p)$$

These offsets modify the original attributes of the gaussians. Afterwards, these gaussians are further deformed by the Rigid Deformer f_r to the observation space. More specifically, for each gaussian x_d, the Rigid deformer predicts the weights to be used with linear blend skinning. If B_b is the transformation of each bone expressed as a 4x4 matrix containing the rotation and translation, then, the total transformation matrix T is given by:

$$T = \sum_{b=1}^{B} f_r(x_d)_b B_b$$

The deformed gaussians are then transformed to the observation space as:

$$\mathbf{x}_o = T\mathbf{x}_d$$

However, this accounts only for the means of the gaussians. The variances should also be transformed. This happens by multplying the rotation matrices of the gaussians with the 3x3 submatrix representating the linear transformation:

$$\mathbf{R}_o = T_{1:3,1:3}\mathbf{R}_d$$

Finally the texture module takes as input the gaussian features, the feature $\mathbf{z}$ predicted by f_{nr}, the latent pose feature and the viewing direction and produces the texture of each gaussian. The rendered image is compared to the ground truth using photometric losses and regularization losses.

Most importantly, the authors introduce a loss that encourages the deformation procedure to be as isometric as possible. This loss is applied both on the gaussian means and the covariances. More specifically, during each iteration, they find for each gaussian its closest k neighbors. They force the gaussians to be similar before and after the deformation by measuring the distances between the gaussian and its neighbors in the canonical and observation space and forcing them to be close:

$$L_{isopos} = \sum_{i=1}^{N} \sum_{j \in N_k(i)} |d(x_c^{(i)}, x_c^{(j)}) - d(x_o^{(i)}, x_o^{(j)})|$$

and in the same way for the covariances:

$$L_{isocov} = \sum_{i=1}^{N} \sum_{j \in N_k(i)} |d(\Sigma_c^{(i)}, \Sigma_c^{(j)}) - d(\Sigma_o^{(i)}, \Sigma_o^{(j)})|$$

3.2. 2DGS Integration

Given the similarities in the interface between 3DGS and 2DGS, it is easy to extend the previous method in order to use the 2DGS framework. We highlight three differences here. First of all, following the original work [3] we randomly initialize the rotations of the disk, instead of using the identity matrix. Moreover, we no longer have covariance matrices to enforce the as isometric as possible loss. However we can enforce it on the RS matrices that define the local frame and the variance of the 2D Gaussians. Finally, we observe that during the rigid transformation, the submatrix $T_{1:3,1:3}$ is not an orthogonal matrix, thus the first 2 columns of $\mathbf{R}_o$ are no longer an orthonoral basis of the local UV space of the gaussians. However, they remain a valid parameterization of a local frame. In our experiments we also propose a method to recitfy $\mathbf{R}_o$ into being a proper rotation matrix. Most importantly, we also add the distortion and normal consistency loss as proposed in [3].

After integrating, we also explore extensions in order to further improve the quality.

Rotation initialization. As mentioned before, to create the gaussians in the canonical space we sample 50k points on the surface of the SMPL mesh in the canonical pose. Since we now have 2D Gaussians, we propose to initilize their rotation so their normal (cross product of the tangential vectors) aligns with the normal at the SMPL mesh.

Rotation rectification. As previously mentioned, after the rigid transformation $\mathbf{R}_o$ is no longer a valid rotation matrix, and its first two columns no longer define an orthonormal base. To this end, we explore the use of SVD in order to find the closest, in terms of frobenious norm, rotation matrix and set this as the rotation of the Guassians.

Laplacian loss. We draw inspiration from the *as isometric as possible* loss and [2] we also explore the use of a *Laplacian loss* where it forces the gaussians to have the same topology in the canonical and observation space. For each gaussian we sample k neighbors and use a uniform weighting sheme.

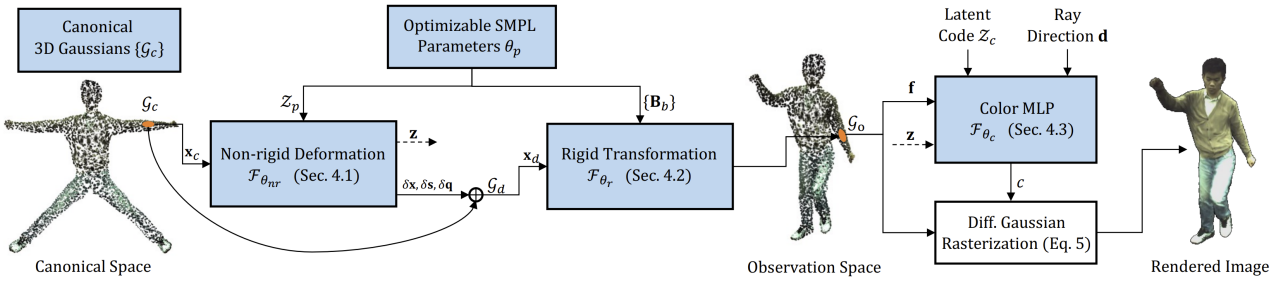


Figure 1. 3DGS Avatar pipeline. Image from [8].

3.3. GOF Integration

To extend 3DGS to GOF, we first compute the opacity fields in the observation space since GOF requires computing ray-Gaussian intersections for each deformed Gaussian in the resulting space rather than in the canonical. Secondly, we use two key regularization losses from the original GOF paper on top of the losses from 3DGS-Avatar, depth distortion and normal consistency losses. The depth distortion loss encourages Gaussians along the same ray to lie close together in depth as:

$$\mathcal{L}_{\text{depth}} = \sum_{i,j} \omega_i \omega_j \left| t_i - t_j \right|,$$

where ω_i denotes the blending weight of the i-th Gaussian and i, j are indices of Gaussians contributing to the ray. On the other hand, the normal consistency loss encourages the estimated normals of Gaussians to align with the image-space depth gradients as:

$$\mathcal{L}_{\text{normal}} = \sum_{i} \omega_i (1 - \mathbf{n}_i^\top \mathbf{N}_i).$$

where $\mathbf{N}$ is the normal estimated by the depth map. We use the loss function introduced in GOF, where the normal of a Gaussian is defined from the ray-Gaussian intersection plane as $\mathbf{n}_i = -\mathbf{R}_k^\top \mathbf{S}_k^{-1} \mathbf{r}_g$, on contrary to 2DGS case where the normal points outwards from the center. For further experiments, we use Laplacian loss for GOF as in the 2DGS integration. In our experiments, we choose coefficients of depth distortion, normal consistency, and Laplacian loss as 1, 0.05, and 1, respectively.

To extract mesh from GOF model, we follow the tetrahedral mesh extraction technique proposed in the GOF paper, where a tetrahedral grid is constructed around the Gaussians' bounding boxes and Marching Tetrahedra [1] is used to extract adaptive meshes from the opacity field level sets.

4. Experiments

4.1. Datasets

We base our experiments on the ZJU-MoCap dataset, a multi-view capture dataset widely used for human performance reconstruction. It consists of synchronized RGB videos captured from 23 cameras, along with background masks, accurate camera calibrations, and per-frame SMPL pose and shape annotations.

Preprocessing. To make the dataset compatible with our Gaussian avatar reconstruction pipeline, we adopt the preprocessing pipeline provided by the ARAH framework. This step converts the raw data into a format that includes all necessary geometry and appearance information for training.

The preprocessing extracts and organizes RGB images and segmentation masks for each frame and camera view. It reconstructs per-frame posed meshes using the SMPL body model and aligns them with the camera extrinsics. In addition, it computes joint transforms, body shapes, and minimal geometry, storing these in a structured format. This enables integration of accurate skeletal pose and shape into neural rendering pipelines.

The final dataset consists of synchronized RGB frames, clean foreground masks, camera intrinsics and extrinsics, and per-frame SMPL-based 3D mesh information, serving as a consistent input source for Gaussian-based avatar methods.

4.2. Evaluation Metrics

To evaluate the quality of the reconstructed avatars, we use both image-based and geometry-based metrics. Image-based metrics assess the fidelity of rendered images compared to ground truth RGB frames:

- **PSNR (Peak Signal-to-Noise Ratio):** Measures the accuracy of the pixel reconstruction, favoring the high-frequency correctness.

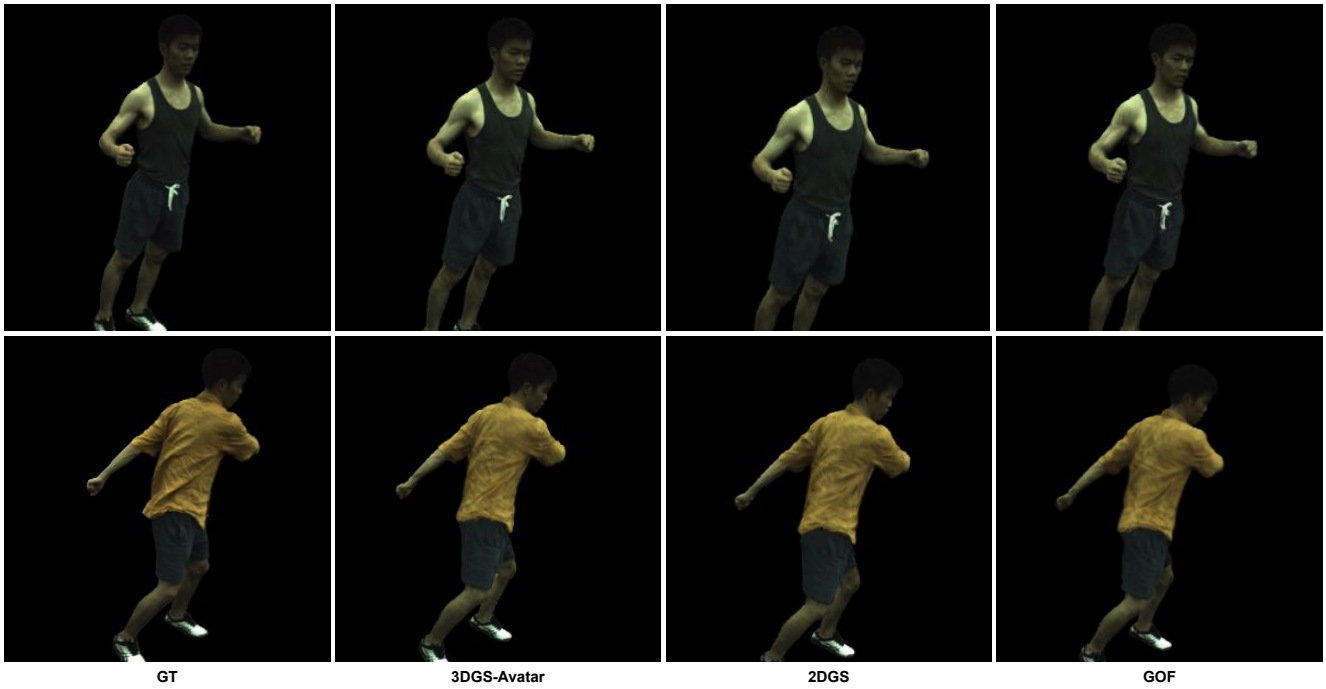


Figure 2. Qualitative comparison on ZJU-MoCap on subject 377 and 393.

- **SSIM (Structural Similarity Index):** Evaluates perceptual similarity by considering luminance, contrast, and structural information of corresponding image patches.
- **LPIPS (Learned Perceptual Image Patch Similarity):** A perceptual metric based on deep features of a pretrained neural network.

Geometry-based metrics evaluate the quality of the extracted mesh:

- **Chamfer Distance (CD):** Measures the average bidirectional distance between points on the predicted mesh and ground truth surface, capturing geometric accuracy.
- **Normal Consistency (NC):** Quantifies how well the surface normals align between the prediction and ground truth, reflecting smoothness and fine detail preservation.

These metrics collectively provide a comprehensive view of both visual plausibility and geometric fidelity in the reconstructed avatars.

4.3. Baselines and Methods

We present our final quantitative results in 1 and our qualitative results in 2. We observe that in term of appearance metrics, all models perform equally. In the next sections we present our experiments.

4.3.1 Baseline & Depth Extraction

Our baseline consists of the 3DGS-avatar method [8]. We will compare our results in terms of appearance and geometry with the results of this model. However, the aforementioned work does not produce depth values that can be used to create the meshes. To this end, we ablate 3 depth extraction methods from 3DGS primitives and select the one that achieves the best results. In the next paragraphs we describe how 3DGS/2DGS methods are used to create meshes, specific details for our case and the results from different depth extraction methods.

Evaluation procedure: In order to evaluate the geometrical results we follow the method proposed by [3]. More specifically, given a 3DGS representation of the scene, we can extract a mesh by rendering depth values from multiple viewpoints and then fusing them using the TSDF integration method from [11]. In our case, we perform this procedure for a number of held out frames (unseen during training) and compare agains pseudo ground truth meshes that are extracted with the method Neus [9].

Camera Setup: Given a frame we need to get depth images from multiple viewpoints. Since the number of real viewpoints is limited (23 cameras in total) we create virtual cameras by modifying the extrinsic parameters of the first test camera. More specifically, we create 100 cameras on a grid on the unit square and then warp them to the surface of the unit sphere. Moreover, we estimate the radius of the scene from the available test cameras and also scale the size

Table 1. Per-subject evaluation metrics on subjects 377 and 393.

Model	377					393				
	PSNR↑	SSIM↑	LPIPS↓	CD↓	NC↑	PSNR↑	SSIM↑	LPIPS↓	CD↓	NC↑
3DGS-Avatar (Paper)	**30.64**	**0.9774**	**20.88**	-	-	28.88	0.9635	35.26	-	-
3DGS-Avatar	30.51	0.9769	21.29	1.58	0.63	28.84	0.9631	35.19	2.57	0.52
2DGS-Avatar	30.59	0.9771	21.51	**1.45**	**0.55**	28.95	0.9636	35.65	4.75	0.53
GOF-Avatar	30.48	0.9747	30.28	2.01	0.46	28.78	0.9622	45.42	5.67	0.26

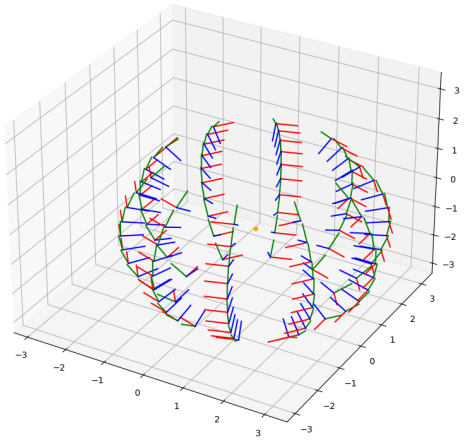


Figure 3. Location of generated cameras for mesh extraction. The orange point indicates the location of the root joint of the human.

of the sphere. The location of the cameras can be seen in 3.

Depth Extraction: The most commonly used depth rendering techinque in 3DGS is the expected depth defined as:

$$D = \sum_i d_i a_i T_i$$

where d_i is the z value of the gaussian after transforming it in the camera space. However, this type of extraction creates floaters. For these reason many authors prefer the median depth. Median depth is the depth of the gaussian that causes the transmittance to fall below 0.5. However, median depth does not alleviate the "holes" that sometimes are created by construction, because we may skip some gaussians before dropping below 0.5. Finally we also modify the method to return the depth of first intersection. This is equivalent to the depth of the gaussian that causes the transmittance to drop below 1.0.

In Table 2 we present our findings regarding depth extraction. More specifically we train the baseline and during extraction we use 3 different rasterizers that implement the three different depth renderings.

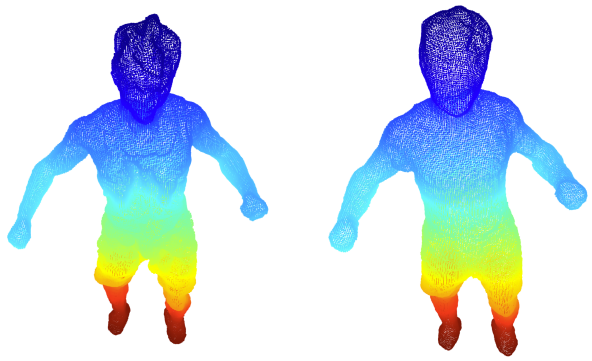


Figure 4. Meshes generated with the median depth (left) and the first intersection depth (right).

Table 2. Different depth extraction methods.

Method	CD↓	NC↑
Expected	4.92	0.52
Median	4.49	0.53
First Intersect.	1.58	0.63

We further investigate the big difference in terms of Chamfer Distance between the first intersection depth and the others. In Fig. 4 we notice that there is a big cavity in the torso of the human that is not visible during image rendering. However, by rendering the depth we see that indeed there is a region with large depth values in the torso. That confirms that the difference in meshes is solely based on the different depth extraction method used. One last observation that is also apparent in the figures is that fusing first-intersection depths tend to create coarser meshes that loose details(e.g. details on the arms); however, achieves better geometric results and we use these results to compare our model to.



Figure 5. Rendered median depth (left) and first intersection depth (right).

4.3.2 2DGS-Avatar

In all our experiments we tuned our hyperparameters based on experiments with subject 377 and then applied the best model on subject 393. In our experiments we use the median depth to implement the two additional losses. In addition, during depth rendering we also use the median depth as suggested by the authors of [3] for a bounded scene. Our best performing model in terms of Chamfer Distance is the one that initializes the rotation matrices to be aligned with the surface normals of the SMPL mesh. Since this is our main objective we choose this to be our final model. However, the model that uses rotation rectification achieves higher normal consistency.

First of all, we performed a grid search over the weights of the *as isometric as possible covariance loss*, *distortion loss* and *normal consistency loss*. In Table 3 we present the results for the geometric metrics only since the photometric losses are very similar. Each triplet represents the three weights with this order.

According to the results, we narrowed down our experiments and remove the *as isometric as possible* covariance loss from the pipeline. Moreover, we selected a weight 0.05 for the *normal consistency* loss. We found it beneficial to increase the delay of the training of the gaussian parameters from 1000 to 2000 iterations, causing a drop of 2mm in terms of chamfer distance.

Ablations. As previously mentioned, we also explore some additional ways to improve the performance. In Table 4 we present a comparison of these approaches.

Table 3. Ablation of loss weights in 2DGS.

Model	CD↓	NC↑
2DGS(0,0,0)	4.78	0.54
2DGS(0,0,0.05)	4.62	**0.60**
2DGS(0,1,0)	6.51	0.53
2DGS(0,1,0.05)	**4.17**	0.58
2DGS(0,10,0)	5.76	0.52
2DGS(0,10,0.05)	5.17	0.60
2DGS(1,0,0)	7.47	0.54
2DGS(1,0,0.05)	7.39	0.54
2DGS(1,1,0)	6.77	0.50
2DGS(1,1,0.05)	7.08	0.55
2DGS(1,10,0)	7.90	0.49
2DGS(1,10,0.05)	7.16	0.54

Table 4. Ablation study of 2DGS variants.

Model	PSNR↑	SSIM↑	LPIPS↓	CD↓	NC↑
2DGS-Avatar	30.59	0.9771	21.51	**1.45**	0.55
w/ random init	30.61	**0.9773**	21.39	1.94	0.63
w/ random rectify	**30.62**	0.9771	21.95	1.90	**0.64**
w/ Laplacian loss	30.50	0.9767	22.18	2.58	0.63

4.3.3 GOF-Avatar

We evaluate the impact of the Laplacian loss on GOF-Avatar performance. As shown in Table 5, using the Laplacian loss leads to improvements in PSNR and SSIM, indicating better pixel-wise and structural reconstruction quality. However, it also results in a higher LPIPS score, suggesting a slight degradation in perceptual similarity.

Table 5. Laplacian loss on GOF.

Model	PSNR↑	SSIM↑	LPIPS↓
GOF-Avatar	30.48	0.9747	30.28
w/ Laplacian Loss	30.51	0.9756	33.36

We also compare TSDF and Marching Tetrahedra mesh extraction algorithms on subject 377 from ZJU-MoCap dataset. As shown in Table 6, Marching Tetrahedra significantly improves the geometric quality of the extracted mesh, achieving a lower CD score. However, due to the rapid growth in the number of Gaussians during GOF optimization, particularly after 5000 steps, we restrict the densification ratio to prevent excessive memory usage. This constraint reduces the number of Gaussians and degrades reconstruction quality.

Table 6. Comparison of TSDF and Marching Tetrahedra in GOF.

Method	CD↓	NC↑
TSDF	5.43	0.45
Marching Tetrahedra	2.01	0.46

5. Conclusion

5.1. Summary of Contributions

In this project, we investigated how geometry-aware Gaussian splatting methods can improve the reconstruction of animatable human avatars from monocular video. Building on the 3DGS-Avatar baseline, we integrated and evaluated two approaches: 2D Gaussian Splatting (2DGS) and Gaussian Opacity Fields (GOF). Moreover, we experimented with different depth rendering techniques and investigated failure cases on mesh extraction. While 2DGS-Avatar achieved marginal improvements in geometric accuracy (e.g., lower Chamfer Distance on subject 377) and slightly better normal consistency than the baseline, it did not outperform the original 3DGS-Avatar in photometric metrics such as PSNR, SSIM, or LPIPS. GOF-Avatar, on the other hand, enabled adaptive mesh extraction through Marching Tetrahedra but suffered from degraded visual and geometric quality, likely due to densification constraints and fewer Gaussians. These results suggest that while geometry-focused extensions can enhance mesh-level detail, they currently trade off performance in image-based fidelity. Further improvements are needed to bridge this gap without compromising rendering quality.

5.2. Limitations and Future Work

First, while 2DGS aligns splats with the mesh surface improves geometric accuracy, the reliance on manually defined initialization and rotation rectification may limit robustness and automation. Furthermore, we observed tradeoffs between surface smoothness and fine detail when using different depth extraction methods, such as median versus first-intersection depth.

The GOF optimization process results in rapid densification of Gaussians, which significantly increases memory consumption and limits the scalability of the method. To manage this, we restrict the number of Gaussians by reducing the densification ratio. However, this constraint leads to coarser reconstructions and a noticeable loss of mesh detail. Moreover, the choice of mesh extraction technique plays a critical role in the final quality whereas TSDF fusion is widely used, our experiments show that Marching Tetrahedra over a tetrahedral grid, as used in GOF, produces more geometrically accurate surfaces. This suggests that the effectiveness of GOF may depend not only on the underlying representation but also on the choice of mesh extraction algorithm.

A general issue regarding this work is the decoupling of mesh extraction with the appearance model. We showcased that the same 3DGS representation can produce meshes of different qualities when changing the way the depth is rendered. Moreover further improvements can come by tuning the TSDF integration hyperparameters. This decoupling, obstructs the authors from drawing certain conlcusion since the imrpovement in geometry may be attributed to the mesh extraction method and not the underlying representation.

6. Contributions of team members

The project tasks were divided as follows: Sotiris Karapiperis was responsible for conducting the 2DGS experiments and 3DGS depth experiments, Emircan Gündoğdu was responsible for the GOF experiments, and Brian Barth implemented the evaluation scripts and utility functions.

References

[1] Akio Koide Akio Doi. An efficient method of triangulating equi-valued surfaces by using tetrahedral cells. *IEICE TRANSACTIONS on Information*, E74-D(1):214–224, January 1991. 4

[2] Thiemo Alldieck, Marcus Magnor, Weipeng Xu, Christian Theobalt, and Gerard Pons-Moll. Video based reconstruction of 3d people models. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 8387–8397, 2018. 3

[3] Binbin Huang, Zehao Yu, Anpei Chen, Andreas Geiger, and Shenghua Gao. 2d gaussian splatting for geometrically accurate radiance fields. In *ACM SIGGRAPH 2024 conference papers*, pages 1–11, 2024. 1, 2, 3, 5, 7

[4] Wei Jiang, Kwang Moo Yi, Golnoosh Samei, Oncel Tuzel, and Anurag Ranjan. Neuman: Neural human radiance field from a single video, 2022. 1

[5] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering, 2023. 1

[6] Muhammed Kocabas, Rick Chang, James Gabriel, Oncel Tuzel, and Anurag Ranjan. Hugs: Human gaussian splats, 2023. 1, 2

[7] Shenhan Qian, Tobias Kirschstein, Liam Schoneveld, Davide Davoli, Simon Giebenhain, and Matthias Nießner. Gaussianavatars: Photorealistic head avatars with rigged 3d gaussians. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20299–20309, 2024. 2

[8] Zhiyin Qian, Shaofei Wang, Marko Mihajlovic, Andreas Geiger, and Siyu Tang. 3dgs-avatar: Animatable avatars via deformable 3d gaussian splatting. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 5020–5030, 2024. 1, 2, 3, 4, 5

[9] Peng Wang, Lingjie Liu, Yuan Liu, Christian Theobalt, Taku Komura, and Wenping Wang. Neus: Learning neural implicit

surfaces by volume rendering for multi-view reconstruction. *arXiv preprint arXiv:2106.10689*, 2021. 5

[10] Zehao Yu, Torsten Sattler, and Andreas Geiger. Gaussian opacity fields: Efficient adaptive surface reconstruction in unbounded scenes. *ACM Transactions on Graphics*, 2024. 1, 2

[11] Qian-Yi Zhou, Jaesik Park, and Vladlen Koltun. Open3d: A modern library for 3d data processing. *arXiv preprint arXiv:1801.09847*, 2018. 2, 5